

1 Introduzione

1.1 Presentazione del problema

Una società informatica vuole realizzare una piattaforma informatica per raccogliere promozioni/offerte, da parte di negozi e supermercati su varie tipologie di prodotti sul territorio nazionale, che potranno essere consultati dai vari clienti della stessa.

La piattaforma deve permettere:

- La registrazione di clienti;
- La registrazione di negozi/supermercati;
- La consultazione delle offerte/promozioni attive da parte dei clienti sui prodotti;
- La possibilità di lasciare voti e commenti sulle offerte da parte dei clienti;
- L'inserimento di nuove offerte da parte dei negozi aderenti;

2 Specifiche

2.1 Specifiche sui dati

Il committente richiede che si gestiscano le seguenti informazioni:

- Per gli *Utenti* : Cognome, Nome, Località, Email, Foto, Username e Password.
- Per i *Negozi* : Nome, Tipologia, Descrizione, Indirizzo (completo), Latitudine, Longitudine, Recapiti Telefonici, Fax , P. iva, Email, Appartenenza ad un Centro Commerciale, Appartenenza ad una Catena di Negozi, Username e Password.
- Per i *Prodotti* : Codice di Identificazione, Nome, Descrizione, Prezzo, Categoria, Sottocategoria e Galleria di Immagini.
- Per ogni *Offerta* : Codice di Identificazione, Prodotto, Data inizio, Data fine, Sconto in Percentuale, Breve Descrizione, Commenti e Voti.

La società si aspetta che nel primo anno di esercizio del sistema questo dovrà gestire:

1. l'anagrafica di circa 10000 utenti (2000 negozi e 8000 utenti);
2. 8000 comuni;
3. circa 20000 offerte e 100000 prodotti.

2.2 Specifiche sulle operazioni

Dopo la registrazione dell'utente e dei negozi sulla piattaforma, si richiede per essa:

- Che la piattaforma mostri soltanto le offerte ancora disponibili;
- Che le offerte scadute vengano automaticamente archiviate;
- Che sia responsabilità dell'amministratore inserire nuovi prodotti su apposita richiesta dei negozi;
- Che sia possibile visualizzare per ogni offerta le votazioni ricevute ed eventuali commenti fatti su di esse;
- Che ogni negozio possa inserire le proprie offerte;
- Che ogni utente possa ricercare le promozioni/offerte in base a diversi criteri quali: la posizione dei negozi più vicini a lui (indirizzo), gli hashtag con i quali le offerte sono state lanciate, la categoria dei prodotti interessati, il nome dei vari negozi.

2.3 Specifiche sugli utenti della base di dati

Per il servizio richiesto, si è scelto l'utilizzo di tre tipologie di utenti:

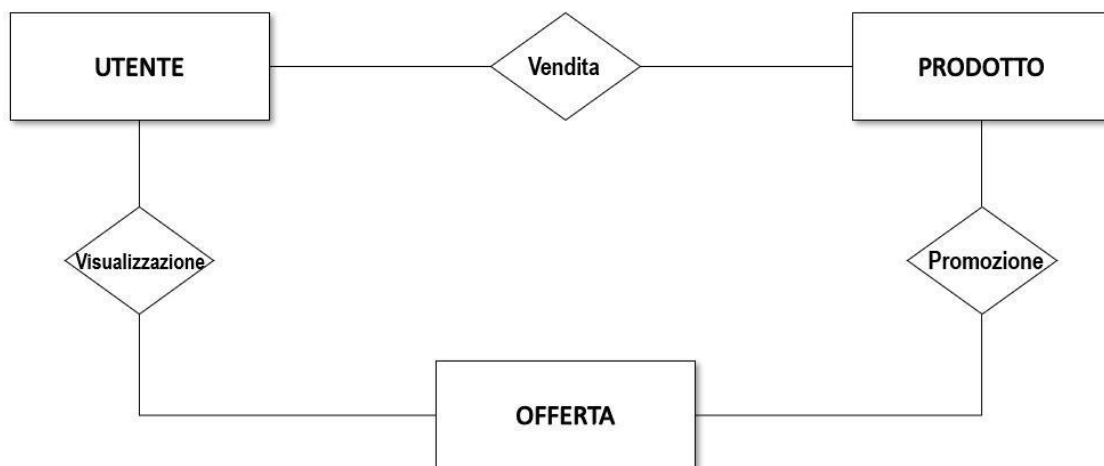
1. Un utente DBA che è responsabile della registrazione degli utenti e dell'inserimento dei nuovi prodotti nella piattaforma.
2. Un utente (cliente) che è in grado solamente di ricercare le offerte, lasciare voti e commenti ed eventualmente usufruirne.
3. Un utente (negozio) che, dopo aver stipulato un contratto, può caricare sulla piattaforma le proprie offerte e richiedere, eventualmente, l'inserimento nel catalogo di nuovi prodotti.

3 Progettazione concettuale

3.1 Schema E/R portante

1. Inizialmente si è definito un modello E/R portante che sottolineasse gli aspetti fondamentali del problema, e solo successivamente si è optato per raffinamenti ed estensioni, inserendo nel dettaglio gli attributi delle varie entità.
2. Nel modello E/R portante, dunque, sono state individuate tre entità chiave: Utente, Prodotto, Offerta.

Vediamo dunque che lo schema portante è il seguente:



3.2 Scherma E/R avanzato

Per poter raffinare ed elaborare il modello E/R portante analizziamo più nel dettaglio le specifiche del progetto.

Una prima cosa di cui rendersi conto è che gli utenti della piattaforma sono di due tipi: **negozi** e **clienti**. È evidente che essi non sono altro che due figlie dell'entità principale utente. Tale gerarchia risulta essere totale e disgiunta. Totale perché un utente, in questo caso, può essere sia un cliente sia un negozio. Disgiunta perché, invece, uno stesso account cliente non può fungere sia da negozio che da cliente.

Infatti, un privato che fa acquisti sulla piattaforma che vuole registrarsi anche come negozio dovrà registrarsi su un account diverso.

In particolare, poi, ogni negozio avrà online sulla piattaforma il proprio catalogo contenente le varie offerte che desidera lanciare con i relativi hashtag associati. Ogni negozio avrà in catalogo da 1 a N offerte, mentre ogni offerta può far parte del catalogo di un solo negozio.

Il fatto che un negozio possa appartenere o meno ad una catena, o in ugual modo ad un centro commerciale, è indicato da un attributo opzionale.

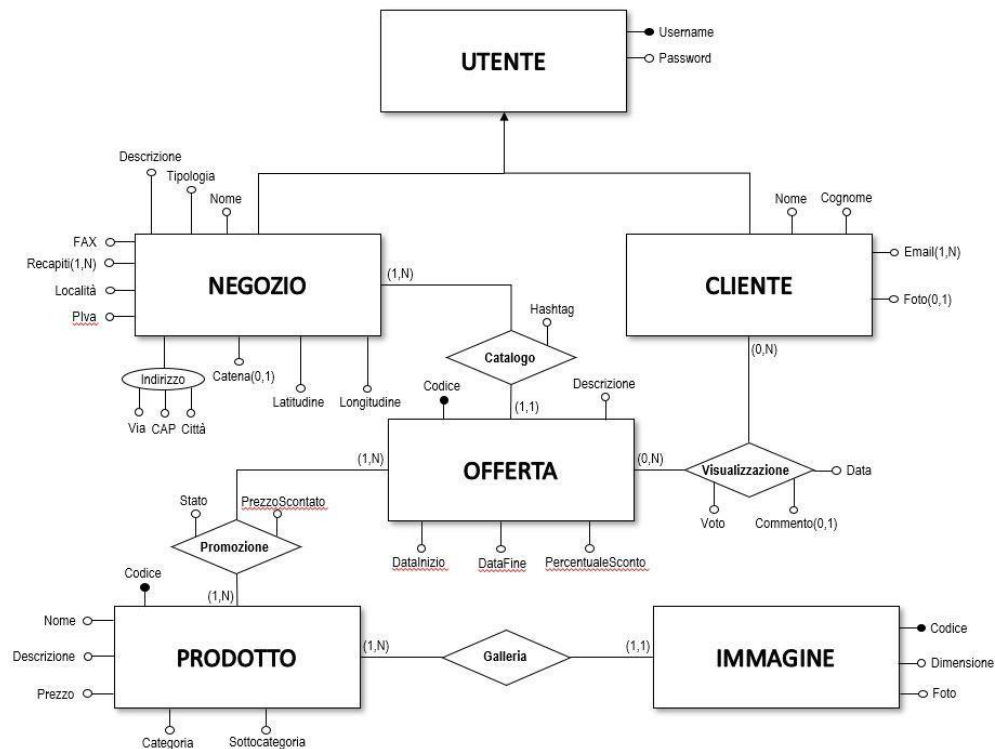
Il cliente registrato può usufruire da 0 a N offerte ed un'offerta può essere usfruita da più clienti o nessuna (perché può non essere molto conveniente).

Il cliente può lasciare, opzionalmente, dei commenti, e allo stesso tempo un utente che usufruisce di un'offerta deve lasciare un voto.

Un'offerta mette in promozione da 1 a N prodotti e lo stesso prodotto può essere messo in promozione da più offerte.

Inoltre, essendo un'offerta identificata univocamente dal codice, non c'è bisogno che la data di inizio partecipi alla chiave primaria. L'offerta inoltre indica per ogni prodotto lo stato di questo ed il prezzo scontato.

I prodotti, inoltre, sono dotati di una galleria di immagini correlata (minimo 1 immagine, fino a N immagini) e la singola immagine può riferirsi ad un solo prodotto. Infine, i negozi possono avere da 1 a N recapiti telefonici, per questo è un attributo multi valore, mentre indirizzo è un attributo composto costituito dalla via, dal numero civico e dal CAP.



3.3 Schema E/R trasformato

A questo punto bisogna trasformare il modello E/R per poterlo poi tradurre nel modello logico-razionale.

Questa fase comprende 3 operazioni:

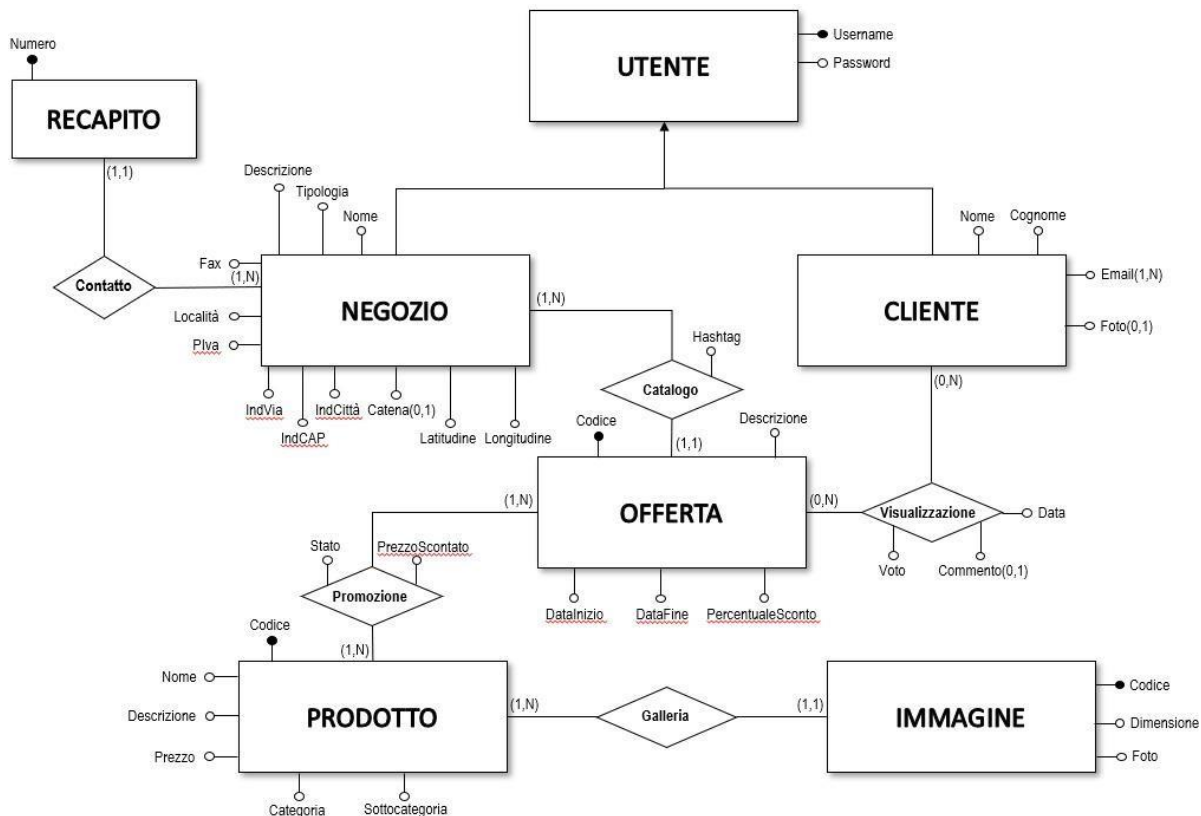
scomporre gli attributi composti nei suoi attributi componenti;

trasformare poi gli attributi multi valore in entità;

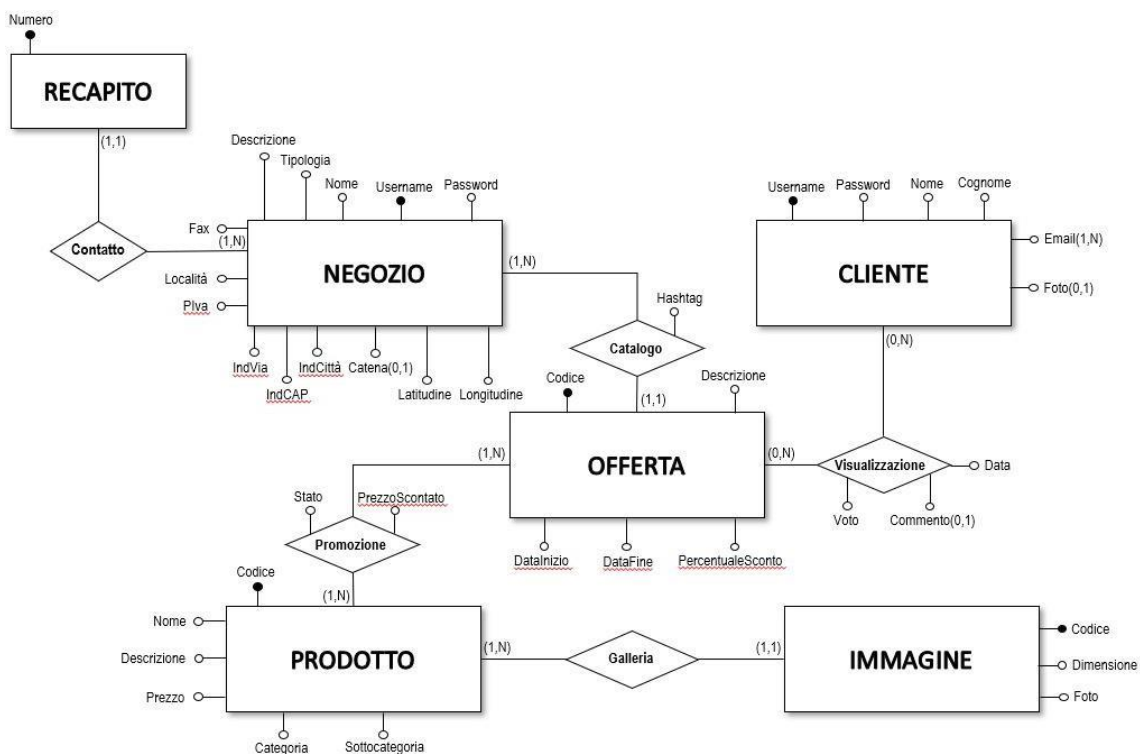
creare un'associazione che la lega con l'entità da cui si è separata e tradurre la gerarchia.

Per quanto riguarda quest'ultima operazione, si è deciso di utilizzare la strategia che prevede di accoppiare l'entità padre nelle entità figlie, poiché le operazioni che sono previste per i negozi ed i clienti sono diverse tra di loro.

Seguendo le prime due operazioni otteniamo un modello E/R di transizione:



Adoperando invece l'ultima operazione, descritta in precedenza, otteniamo il vero e proprio modello E/R trasformato:



4 Progettazione logica

4.1 Dal modello E/R a quello logico-concettuale

Ora, per poter passare al modello logico-relazionale, dal modello E/R trasformato, è necessario seguire delle specifiche regole di traduzione per tradurre entità e associazioni. Sono state create le relazioni (con i nomi al plurale delle entità) aventi come chiavi primarie il proprio attributo primario.

In particolare:

- l'associazione **CONTATTO** è stata tradotta inserendo all'interno della tabella **RECAPITI** l'attributo Username che referencia Username di **NEGOZI** attraverso una chiave esterna;
- l'associazione **GALLERIA** è stata introdotta inserendo all'interno della tabella **IMMAGINI** l'attributo CodProdotto che referencia Codice di **PRODOTTI** attraverso una chiave esterna;
- l'associazione **PROMOZIONE** è stata tradotta come una tabella che ha una chiave primaria composta da due attributi CodProdotto e CodOfferte che sono anche chiavi esterne per **PRODOTTI** e **OFFERTE**, ed in più ha come altro campo l'attributo Stato, PrezzoScontato.
- l'associazione **CATALOGO** è stata tradotta inserendo all'interno della tabella **OFFERTE** l'attributo UsernameNegozio che referencia Username di **NEGOZI** attraverso una chiave esterna e l'attributo Hashtag è stato inglobato in **OFFERTE**.
- l'associazione **VISUALIZZAZIONI** è stata tradotta come una tabella che ha una chiave primaria composta da due attributi Username e CodOfferte che sono anche chiavi esterne per **UTENTI** e **OFFERTE** ed in più ha come campi gli attributi Data, Voto e Commento.

4.2 Schema logico finale

Il risultato di quanto detto è il seguente modello logico-relazione:

CLIENTI(Username, Password, Nome, Cognome, Email, Foto,);

NEGOZI(Username, Password, Nome, Tipologia, Descrizione, IndCap, IndVia, IndCittà, Catena, Piva, FAX, Latitudine, Longitudine, Località);

RECAPITI(Numero, UsernameNegozio:NEGOZI);

OFFERTE(Codice, UsernameNegozio:NEGOZI, DataInizio, DataFine, PercentualeSconto, Descrizione, Hashtag);

PRODOTTI(Codice, Nome, Descrizione, Categoria, Sottocategoria, Prezzo);

IMMAGINI(Codice, Dimensione, Foto, CodProdotto:PRODOTTI);

PROMOZIONI(CodProdotto:PRODOTTI, CodOfferte:OFFERTE, Stato, PrezzoScontato);

VISUALIZZAZIONI(UsernameCliente:CLIENTI, CodOfferte:OFFERTE, Data, Voto, Commento);

5 Progettazione fisica

5.1 Premesse

Arrivati a questo punto si vuole procedere con la progettazione fisica della base di dati, che significa implementare il suddetto schema logico- relazione.

Per poter fare ciò è necessario:

- effettuare una corretta partizione logica dello storage della base dati;
- creare l'utente della base di dati (DBA); creare le tabelle della base di dati e aggiungervi tutti i vincoli necessari;
- creare query per un'analisi statistica della base di dati;
- creare il ruolo 'cliente' e 'negoziante';
- definire le politiche per la gestione della concorrenza e dell'affidabilità.

Per un corretto funzionamento della base di dati è stato necessario aggiungere un'ulteriore tabella ad essa: la tabella STORICO. Essa ha in sé tutte le informazioni delle offerte archiviate.

5.2 Tipi di dato

Determinata la quantità di informazioni che verranno immagazzinate nella Base di Dati, e ultimata la progettazione logica, si è passati a determinare i tipi di dato utilizzati nella creazione delle tabelle. In particolare, sono stati utilizzati i seguenti tipi:

- **CHAR(N)** : è stato utilizzato il tipo char per disporre di un range di caratteri alfanumerici che permettono di comporre codici identificativi a lunghezza fissa completi, e non solo. Esso prevede in ORACLE una collocazione di N byte per N caratteri alfanumerici.
- **VARCHAR2(N)**: è stato utilizzato il tipo varchar2 per gli attributi di tipo testuale a lunghezza variabile che sono più propensi a variazioni (come INDIRIZZO,NOME ,COGNOME ...). Il tipo varchar2

prevede in ORACLE una collocazione di N byte per N caratteri testuali entro la dimensione dei 4000 byte per valore.

- **NUMBER:** dato l'utilizzo di codici identificativi numerici per la loro gestione tramite sequenze, è stato utilizzato il tipo Number per questi campi. Esso prevede in ORACLE una collocazione di $(N/2)+2$ byte per N cifre.

- **DATE:** è stato utilizzato il tipo Date negli attributi di tipo data. Esso prevede in ORACLE una collocazione di 7byte.

- **CLOB:** è stato utilizzato il tipo Clob per attributi di tipo 'descrizione' per permettere testi anche molto lunghi (fino a 4gb) senza limitarli. Esso prevede in ORACLE una collocazione di 4 byte relativi ai soli puntatori, poiché questi oggetti sono gestiti da un apposito tablespace.

5.3 Dimensionamento tabelle e tablespace

RECAPITI		
Numero	NUMBER(10)	7 BYTE
UsernameNegozio	VARCHAR2(20)	20 BYTE

Totale=27*3000 (ipotesi che almeno metà hanno due recapiti) = 0.081 KB = 81 KB

PRODOTTI		
Codice	VARCHAR2(7)	7 BYTE
Nome	VARCHAR2(15)	15 BYTE
Descrizione	CLOB	4 BYTE
Categoria	VARCHAR2(20)	20 BYTE
Sottocategoria	VARCHAR2(20)	20 BYTE
Prezzo	NUMBER(8)	6 BYTE

Totale=72*100000 prodotti = 7.2 MB + TOLLERANZA = 9MB

OFFERTE		
Codice	VARCHAR2(7)	7 BYTE
DataInizio	DATE	7 BYTE
DataFine	DATE	7 BYTE
PercentualeSconto	NUMBER(3)	3.5 BYTE
Descrizione	VARCHAR2(150)	150 BYTE
Hashtag	VARCHAR2(20)	20 BYTE
UsernameNegozio	VARCHAR2(20)	20 BYTE

Totale = 214.5*20000 = 4.29 MB + TOLLERANZA = 6 MB

IMMAGINI		
Codice	VARCHAR2(7)	7 BYTE
Dimensione	NUMBER(4)	4 BYTE
Foto	BLOB	4 BYTE
CodProdotto	VARCHAR2(7)	7 BYTE

Totale = $22 * 100000 = 2.2 \text{ MB}$ + TOLLERANZA = 3MB

NEGOZI		
Username	VARCHAR2(20)	20 BYTE
Password	VARCHAR2(20)	20 BYTE
Nome	VARCHAR2(15)	15 BYTE
Tipologia	VARCHAR2(30)	30 BYTE
Descrizione	CLOB	4 BYTE
IndVia	VARCHAR2(30)	30 BYTE
IndCap	NUMBER(5)	4.5 BYTE
IndCittà	VARCHAR2(30)	30 BYTE
Località	VARCHAR2(30)	30 BYTE
Catena	CHAR(2)	2 BYTE
Plva	NUMBER(11)	7.5 BYTE
FAX	VARCHAR2(30)	30 BYTE
Latitudine	VARCHAR2(12)	12 BYTE
Longitudine	VARCHAR2(12)	12 BYTE

Totale = $247 * 2000 = 494 \text{ KB}$ + TOLLERANZA = 1MB

CLIENTI		
Username	VARCHAR2(20)	20 BYTE
Password	VARCHAR2(20)	20 BYTE
Nome	VARCHAR2(15)	15 BYTE
Cognome	VARCHAR2(15)	15 BYTE
Email	VARCHAR2(30)	30 BYTE
Foto	BLOB	4 BYTE

Totale = $104 \cdot 8000 = 0.832 \text{ MB}$ + TOLLERANZA = 1 MB

PROMOZIONI		
CodProdotto	VARCHAR2(7)	7 BYTE
CodOfferte	VARCHAR2(7)	7 BYTE
Stato	VARCHAR2(3)	3 BYTE
PrezzoScontato	NUMBER(8)	6 BYTE

Totale = $23 \cdot 50000$ (ipotesi di promozioni attive nel corso di 1) = 1.15 MB + TOLLERANZA = 2 MB

VISUALIZZAZIONI		
UsernameCliente	VARCHAR2(20)	20 BYTE
CodOfferte	VARCHAR2(7)	7 BYTE
Data	DATE	7 BYTE
Voto	NUMBER(2)	3 BYTE
Commento	CLOB	4 BYTE

Totale = $41 \cdot 50000$ (ipotesi che ogni cliente fruisce di 25 offerte per anno) = 2.05 MB + TOLLERANZA = 3 MB

STORICO		
Codice	VARCHAR2(20)	20 BYTE
NomeProdotto	VARCHAR2(15)	15 BYTE
NomeNegozio	VARCHAR2(15)	15 BYTE
DataInizio	DATE	7 BYTE
DataFine	DATE	7 BYTE
Voto	NUMBER(2)	3 BYTE

Totale = 67*50000= 3.350 MB + TOLLERANZA = 6 MB

Dimensionate le tabelle, si procede a questo punto a dimensionare i tablespace:

Ricordiamo che un *tablespace* è la suddivisione logica del database che contiene gli oggetti della base di dati.

- sg_ts 300 MB, contiene tutte le tabelle della base di dati.
- sg_foto 250 MB, contiene tutte le immagini della base di dati, sia quelle relative ai prodotti
- sg_feedback 50MB, che contiene i commenti dei feedback.
- sg_descrizioni 25MB, che contiene le descrizioni, relative ai prodotti e ai negozi.

Il codice della creazione dei tablespace è il seguente:

```
create tablespace sg_ts datafile 'C:\app\39351\product\18.0.0\oradata\XE\XEPDB1\sg_ts.dbf' size 300M;
create tablespace sg_foto_ts datafile 'C:\app\39351\product\18.0.0\oradata\XE\XEPDB1\sg_foto_ts.dbf' size 250M;
create tablespace sg_feedback_ts datafile 'C:\app\39351\product\18.0.0\oradata\XE\XEPDB1\sg_feedback_ts.dbf' size 50M;
create tablespace sg_descrizioni_ts datafile 'C:\app\39351\product\18.0.0\oradata\XE\XEPDB1\sg_descrizioni_ts.dbf' size 25M;
```

5.4 Creazione dell'utente DBA, delle tabelle e delle sequenze

Prima di effettuare ogni altra operazione, è necessaria la creazione di un DBA del tablespace appena definito, in modo da poter operare sul database come amministratore.

```
create user sg_dba default tablespace sg_ts identified by 1234;  
grant connect, resource, unlimited tablespace to sg_dba;
```

5.5 Creazione delle tabelle

Procediamo ora alla creazione delle tabelle:

```
CREATE TABLE CLIENTI  
(  
  Username VARCHAR2(20),  
  Password VARCHAR2(20) NOT NULL,  
  Nome VARCHAR2(15) NOT NULL,  
  Cognome VARCHAR2(15) NOT NULL,  
  Email VARCHAR2(30) NOT NULL,  
  Foto CLOB,  
  CONSTRAINT PK_CLIENTI PRIMARY KEY (Username)  
)  
LOB (Foto) STORE AS FOTOCLIENTI(TABLESPACE sg_foto_ts)  
STORAGE (INITIAL 1 M);
```

```
CREATE TABLE RECAPITI  
(  
  Numero NUMBER(10),  
  UsernameNegozio VARCHAR2(20),  
  CONSTRAINT PK_RECAPITI PRIMARY KEY (Numero)  
)  
STORAGE (INITIAL 81 K);
```

```
CREATE TABLE PRODOTTI
(
Codice VARCHAR2(7),
Nome VARCHAR2(15) NOT NULL,
Descrizione CLOB,
Categoria VARCHAR2(20) NOT NULL,
Sottocategoria VARCHAR2(20) NOT NULL,
Prezzo NUMBER(8) NOT NULL,
CONSTRAINT PK_PRODOTTI PRIMARY KEY (Codice)
)
LOB (Descrizione) STORE AS DESCPRODOTTI (TABLESPACE sg_descrizioni_ts)
STORAGE (INITIAL 9 M);
```

```
CREATE TABLE OFFERTE
(
Codice VARCHAR2(7),
DataInizio DATE NOT NULL,
DataFine DATE NOT NULL,
PercentualeSconto NUMBER(3) NOT NULL,
Descrizione VARCHAR2(150) NOT NULL,
Hashtag VARCHAR2(20),
UsernameNegozio VARCHAR2(20),
CONSTRAINT PK_OFFERTE PRIMARY KEY (Codice)
)
STORAGE (INITIAL 6 M);
```

```
CREATE TABLE VISUALIZZAZIONI
(
UsernameCliente VARCHAR2(20),
CodOfferte VARCHAR2(7),
Data DATE NOT NULL,
Voto NUMBER(2) NOT NULL,
Commento CLOB,
CONSTRAINT PK_VISUALIZZAZIONI PRIMARY KEY(UsernameCliente,CodOfferte)
)
LOB (Commento) STORE AS FEEDBACKS (TABLESPACE sg_feedback_ts)
STORAGE (INITIAL 3 M);
```

```

CREATE TABLE IMMAGINI
(
Codice VARCHAR2(7),
Dimensione NUMBER(4),
Foto CLOB,
CodProdotto VARCHAR2(7) NOT NULL,
CONSTRAINT PK_IMMAGINI PRIMARY KEY (Codice)
)
LOB (Foto) STORE AS FOTOPROD (TABLESPACE sg_foto_ts)
STORAGE (INITIAL 3 M);

```

```

CREATE TABLE NEGOZI
(
Username VARCHAR2(20),
Password VARCHAR2(20),
Nome VARCHAR2(15) NOT NULL,
Tipologia VARCHAR2(30) NOT NULL,
Descrizione CLOB,
IndVia VARCHAR2(30) NOT NULL,
IndCap NUMBER(5) NOT NULL,
IndCitta VARCHAR2(30) NOT NULL,
Localita VARCHAR2(30) NOT NULL,
Catena CHAR(2) DEFAULT 'NO',
PIva NUMBER(11) NOT NULL,
FAX VARCHAR2(30) NOT NULL,
Latitudine VARCHAR2(12) NOT NULL,
Longitudine VARCHAR2(12) NOT NULL,
CONSTRAINT PK_NEGOZI PRIMARY KEY(Username),
CONSTRAINT CC_NEGOZI CHECK (CATENA = 'SI' OR CATENA='NO')
)
LOB (Descrizione) STORE AS DESCNEGOZI (TABLESPACE sg_descrizioni_ts)
STORAGE(INITIAL 1 M);

```



```

CREATE TABLE PROMOZIONI
(
CodProdotto VARCHAR2(7),
CodOfferte VARCHAR2(7),
Stato VARCHAR2(3) NOT NULL,
PrezzoScontato NUMBER(8) NOT NULL,
CONSTRAINT PK_PROMOZIONI PRIMARY KEY(CodProdotto,CodOfferte)
)
STORAGE (INITIAL 2 M);

```

```

CREATE TABLE STORICO
(
Codice VARCHAR2(20),
NomeProdotto VARCHAR2(15) NOT NULL,
NomeNegozio VARCHAR2(15) NOT NULL,
DataInizio DATE NOT NULL,
DataFine DATE NOT NULL,
Voto NUMBER(2) NOT NULL,
CONSTRAINT PK_STORICO PRIMARY KEY (Codice)
)
STORAGE (INITIAL 6 M);

```

Creiamo ora, in aggiunta a queste tabelle, delle sequenze che verranno memorizzate come oggetti sulla base di dati e permetteranno di inserire automaticamente dei valori nelle chiavi primarie per ogni tabella dotata di codice:

```

CREATE SEQUENCE Contatore_PRODOTTI;
CREATE SEQUENCE Contatore_OFFERTE;
CREATE SEQUENCE Contatore_IMMAGINI;
CREATE SEQUENCE Contatore_STORICO;

```

5.6 Aggiunta chiavi esterne ed altri vincoli

Le scelte per le politiche di reazione sono state le seguenti:

- Se un negozio viene eliminato, i suoi recapiti vengono eliminati.
- Se un negozio viene eliminato, anche le sue offerte vengono eliminate.

- Se un prodotto viene eliminato, anche le immagini relative vengono eliminate.
- Se un prodotto viene eliminato, anche le promozioni a cui partecipa vengono eliminate.
- Se un'offerta viene eliminata, anche le promozioni a cui si riferisce vengono eliminate.
- Se un cliente viene eliminato, anche le sue visualizzazioni vengono eliminate.
- Se un'offerta viene eliminata, anche i dettagli di chi vi ha usufruito vengono eliminati.
- Se un negozio viene eliminato, anche le sue offerte archiviate vengono eliminate.
- I voti che si possono attribuire devono essere compresi tra 1 e 10.
- Gli hashtag devono iniziare tutti con '#'.
- La data di fine offerta non può essere precedente a quella di inizio.
- La dimensione di un'immagine deve essere maggiore di 0.
- I prezzi originali e scontati dei prodotti devono essere maggiori di 0.
- La percentuale di sconto deve essere compresa tra il 5% ed il 95%.

```

ALTER TABLE RECAPITI ADD CONSTRAINT FK_RECAPITI FOREIGN KEY (UsernameNegozio) REFERENCES NEGOZI (Username) ON DELETE CASCADE;
ALTER TABLE OFFERTE ADD CONSTRAINT FK_OFFERTE FOREIGN KEY (UsernameNegozio) REFERENCES NEGOZI (Username) ON DELETE CASCADE;
ALTER TABLE OFFERTE ADD CONSTRAINT CK1_OFFERTE CHECK (PercentualeSconto >= 5 AND PercentualeSconto <=95);
ALTER TABLE IMMAGINI ADD CONSTRAINT FK_IMMAGINI FOREIGN KEY (CodProdotto) REFERENCES PRODOTTI (Codice) ON DELETE CASCADE;
ALTER TABLE PROMOZIONI ADD CONSTRAINT FK1_PROMOZIONI FOREIGN KEY (CodProdotto) REFERENCES PRODOTTI (Codice) ON DELETE CASCADE;
ALTER TABLE PROMOZIONI ADD CONSTRAINT FK2_PROMOZIONI FOREIGN KEY (CodOfferte) REFERENCES OFFERTE (Codice) ON DELETE CASCADE;
ALTER TABLE PROMOZIONI ADD CONSTRAINT CK1_PROMOZIONI CHECK (PrezzoScontato>0);
ALTER TABLE PRODOTTI ADD CONSTRAINT CK1_PRODOTTI CHECK (Prezzo>0);
ALTER TABLE VISUALIZZAZIONI ADD CONSTRAINT FK1_VISUALIZZAZIONI FOREIGN KEY (UsernameCliente) REFERENCES CLIENTI (Username) ON DELETE CASCADE;
ALTER TABLE VISUALIZZAZIONI ADD CONSTRAINT FK2_VISUALIZZAZIONI FOREIGN KEY (CodOfferte) REFERENCES OFFERTE (Codice) ON DELETE CASCADE;
ALTER TABLE STORICO ADD CONSTRAINT FK_STORICO FOREIGN KEY (NomeNegozio) REFERENCES NEGOZI (Username) ON DELETE CASCADE;
ALTER TABLE VISUALIZZAZIONI ADD CONSTRAINT CK_VISUALIZZAZIONI CHECK (Voto>=1 AND Voto<=10);
ALTER TABLE OFFERTE ADD CONSTRAINT CK2_OFFERTE CHECK (Hashtag LIKE '#%');
ALTER TABLE OFFERTE ADD CONSTRAINT CK3_OFFERTE CHECK (DataFine>DataInizio);
ALTER TABLE IMMAGINI ADD CONSTRAINT CK_IMMAGINI CHECK (Dimensione>0);

```

QUERY

Nel data base le informazioni sono organizzate in una struttura logica che permette di accedere con facilità ad ogni dato. Il modo per accedere a questi dati è la *query*. La query viene scritta in un linguaggio di interrogazione.

Query con offerte raggruppate per località:

```
SELECT N.LOCALITA,O.CODICE  
FROM NEGOZI N, OFFERTE O  
WHERE N.USERNAME=O.USERNAMENEGOZIO  
GROUP BY N.LOCALITA,O.CODICE  
ORDER BY N.LOCALITA;
```

Query con offerte raggruppate per categoria:

```
SELECT P.CATEGORIA,O.CODICE  
FROM OFFERTE O, PRODOTTI P, PROMOZIONI PR  
WHERE PR.CODOFFERTE=O.CODICE AND PR.CODPRODOTTO=P.CODICE  
GROUP BY P.CATEGORIA,O.CODICE
```

Query che restituisce il nome del prodotto più venduto:

```
SELECT V.NOMEPRODOTTO  
FROM VENDITE V  
WHERE V.N_VENDITE=(SELECT MAX(V1.N_VENDITE)  
                    FROM VENDITE V1)
```

Query che restituisce l'offerta più votata:

```
SELECT V.CODICE_OFFERTA  
FROM VOTAZIONI V  
WHERE V.SOMMA_VOTI=(SELECT MAX(V1.SOMMA_VOTI)  
                    FROM VOTAZIONI V1)
```

Query che restituisce l'offerta con più successo:

```
SELECT U.CODICE, U.NUM_OFFERTE
FROM UTILIZZO_OFFERTA U
WHERE U.NUM_OFFERTE=(SELECT MAX(U1.NUM_OFFERTE)
                      FROM UTILIZZO_OFFERTA U1)
```

Query che restituisce l'# più utilizzato:

```
SELECT H.TAG
FROM HASHTAG H
WHERE H.NUMERO_TAG=(SELECT MAX(H1.NUMERO_TAG)
                    FROM HASHTAG H1)
```

Query che restituisce l'offerta con il maggiore sconto:

```
SELECT O.PERCENTUALESCONTO
FROM OFFERTE O
WHERE O.PERCENTUALESCONTO>=ALL(SELECT O1.PERCENTUALESCONTO
                                FROM OFFERTE O1)
```

5.10 Definizione delle politiche di sicurezza

Oltre l'utente sg_dba, con cui si è operato fino ad ora, con la base di dati, è necessario definire altri due ruoli per chi invece deve utilizzare la base di dati attraverso l'applicazione web. In particolare:

- l'utente sg_negozio deve poter inserire le proprie offerte, modificarle a, ed eventualmente cancellarle.
- l'utente sg_clienti deve poter guardare le varie offerte e cercarle in base a diversi criteri ed eventualmente deve poterne usufruire

```
CREATE ROLE sg_negozio;
GRANT CONNECT TO sg_negozio;
GRANT SELECT ON OFFERTE TO sg_negozio;
GRANT INSERT, UPDATE, DELETE ON OFFERTE TO sg_negozio;
GRANT INSERT, UPDATE, DELETE ON RECAPITI TO sg_negozio;

CREATE ROLE sg_clienti;
GRANT CONNECT TO sg_clienti;
GRANT SELECT ON OFFERTE TO sg_clienti;
GRANT SELECT ON NEGOZI TO sg_clienti;
GRANT INSERT, UPDATE, DELETE ON VISUALIZZAZIONI TO sg_clienti;
```

5.11 Gestione della concorrenza

Per evitare la presenza di anomalie che possono portare a malfunzionamenti che possono compromettere il guadagno economico di un'azienda, si è scelto come di consueto il protocollo 2PL stretto (Two Phase Locking). Esso prevede che ogni transazione agisca in due fasi (una crescente e una decrescente): nella fase crescente essa acquisisce tutti i lock sugli oggetti su cui dovrà fare lettura/scrittura, nella fase decrescente (dopo aver fatto le operazioni in programma) rilascerà questi lock rendendo di nuovo gli oggetti della base di dati liberi. Successivamente si è scelto il livello di isolamento: REPEATABLE READ. Questo garantisce che se un utente consulta due volte un'offerta della nostra base di dati, vedrà sempre lo stesso risultato.

5.12 Gestione dell'affidabilità

Durante il periodo di funzionamento della piattaforma, potrebbe capitare che a causa di imprevisti o guasti più o meno gravi, i dati vengano persi. Quindi, è necessario prevenire in anticipo il problema adottando delle politiche di gestione dell'affidabilità per poter salvaguardare questi dati. Le tecniche di recovery scelte differiscono in base alla tipologia dei guasti che si possono verificare:

- in caso di guasti soft (es. errori di programma, crash di sistema, caduta di tensione), si utilizza la ripresa a caldo, secondo la quale il corretto funzionamento del database viene ripristinato dall'ultimo checkpoint e, a partire da esso, vengono stabilite le operazioni da rifare sulla base di dati e quelle da disfare;
- in casi di guasti hard (es. guasti sulla memoria di massa, memoria centrale, memoria secondaria), si utilizza la ripresa a freddo, secondo la quale il corretto funzionamento del database viene ripristinato a partire dal backup accedendo al più recente record di dump del log, successivamente si eseguono tutte le operazioni registrate sul log relativamente alla parte deteriorata riportandosi all'istante antecedente al guasto, ed infine viene effettuata una ripresa a caldo.